

# Fleet Health Persistence Migration and Local Simulation Testing

---

**Document type:** Engineering migration and validation report

**Branch:** `dependency_fix`

**Validation date:** 20 September 2026

**Repositories validated:** Common, Robot, Sensor, shared interfaces, and Simulation

## 1. Purpose

This document records the migration of fleet-health persistence to the final PostgreSQL-backed design and the local tests performed after the migration. It also records two runtime corrections found while validating the split five-repository simulation stack.

The objective of the test was to verify package integration, Gazebo hardware simulation, camera data, robot control, MoveIt ownership, ROS interfaces, scan startup, and the PostgreSQL fleet-health store. The test did not change the configured perception safety threshold simply to force a successful scan result.

## 2. Migration Summary

### 2.1 Fleet-health persistence

Fleet-health signal persistence was changed from the transitional SQLite-compatible implementation to PostgreSQL.

The migrated design now provides:

- a dedicated `fleet_health` PostgreSQL migration set;
- PostgreSQL storage for current conditions;
- a persistent event outbox with idempotency keys, retry state, and acknowledgement state;
- storage for the latest runtime snapshot;
- PostgreSQL conflict handling for condition updates and duplicate event protection;
- PostgreSQL-backed access for the fleet-health signal monitor and terminal viewer; and
- named-column access for aggregate query results in the fleet-health and communication queues.

The state-manager migrations were converted from one module into a migration package. This allows the existing service migrations and the dedicated fleet-health migration to coexist under the same persistence namespace.

### 2.2 Common package installation

The Common package still attempted to install directories that had been removed during repository separation. The stale `include/` and `srv/` entries were removed from `as_edge_common/CMakeLists.txt`.

This correction allows the Common package to complete the CMake installation stage in a clean workspace.

### 2.3 Robot simulation startup corrections

Local simulation exposed two Robot package startup issues:

1. The upstream Doosan Gazebo ROS-control macro accepts a parameter named `rate`. The scanner xacro passed `update_rate`. The Gazebo call now maps the existing update-rate value to the expected `rate` parameter. The real-hardware macro was not changed.
2. The hand-eye calibration server and robot hardware-status publisher were installed as ROS executables but did not have executable file permissions. Their executable permissions were restored.

## 2.4 Sensor package

The removed Sensor `camera_pipeline.launch.py` file was not restored. Camera simulation continues to be owned by the Simulation package, while the Sensor repository remains limited to camera-specific runtime support.

## 3. Local Test Environment

The five repositories were tested together:

| Repository                                | Package or responsibility                                   |
|-------------------------------------------|-------------------------------------------------------------|
| <code>Dyne-vision-common</code>           | <code>as_edge_common</code> and ArmX-E application services |
| <code>Dyne-vision-robot</code>            | <code>as_edge_robot_doosan_a0912</code>                     |
| <code>Dyne-vision-sensor</code>           | <code>as_edge_camera_intel_d405</code>                      |
| <code>adaptive_scanning_interfaces</code> | Shared ROS message, service, and action contracts           |
| <code>adaptive_scanning_simulation</code> | <code>as_sim</code> , Gazebo assets, and simulation runner  |

An installed Doosan ROS 2 overlay was sourced as an external vendor dependency. It supplies `dss_description2` and related Doosan packages and is not one of the five project repositories.

## 4. Simulation Path Commands

### 4.1 Stop any earlier simulation

```
cd /home/vishwas/Projects/clients/triple_infotech/adaptive_scanning_simulation

ADAPTIVE_SCANNING_WS_ROOT=/tmp/adaptive-scanning-ws.fkcXXi \
./sim/sim.sh down
```

### 4.2 Create an isolated five-repository workspace

The repositories in the project directory are siblings rather than children of a dedicated ROS `src/` directory. A temporary isolated workspace was therefore used to prevent Colcon from discovering legacy monorepos and unrelated ROS packages.

```

sim_ws=$(mktemp -d /tmp/adaptive-scanning-ws.XXXXXX)
mkdir -p "$sim_ws/src"

ln -s /home/vishwas/Projects/clients/triple_infotech/Dyne-vision-common \
"$sim_ws/src/Dyne-vision-common"
ln -s /home/vishwas/Projects/clients/triple_infotech/Dyne-vision-robot \
"$sim_ws/src/Dyne-vision-robot"
ln -s /home/vishwas/Projects/clients/triple_infotech/Dyne-vision-sensor \
"$sim_ws/src/Dyne-vision-sensor"
ln -s /home/vishwas/Projects/clients/triple_infotech/adaptive_scanning_interfaces \
"$sim_ws/src/adaptive_scanning_interfaces"
ln -s /home/vishwas/Projects/clients/triple_infotech/adaptive_scanning_simulation \
"$sim_ws/src/adaptive_scanning_simulation"

```

The generated workspace used for the recorded test was:

```
/tmp/adaptive-scanning-ws.fkcXXi
```

### 4.3 Build all five packages

```

cd /home/vishwas/Projects/clients/triple_infotech/adaptive_scanning_simulation

ADAPTIVE_SCANNING_WS_ROOT=/tmp/adaptive-scanning-ws.fkcXXi \
./sim/sim.sh build

```

The command invokes the following package build scope:

```

colcon build --symlink-install \
--packages-up-to \
as_edge_common \
as_edge_robot_doosan_a0912 \
as_edge_camera_intel_d405 \
as_sim

```

`adaptive_scanning_interfaces` is included automatically as a dependency.

### 4.4 Source ROS and the external Doosan overlay

```

set +u
source /opt/ros/jazzy/setup.bash
source /home/vishwas/Projects/clients/triple_infotech/install/setup.bash
set -u

```

### 4.5 Run the integrated simulation

The normal-scale test used dataset part 10 and headless Gazebo:

```

cd /home/vishwas/Projects/clients/triple_infotech/adaptive_scanning_simulation

ADAPTIVE_SCANNING_WS_ROOT=/tmp/adaptive-scanning-ws.fkcXXi \
SIM_PART=10 \
SIM_GZ_GUI=false \
./sim/sim.sh up

```

A diagnostic rerun used a larger simulated object scale to determine whether the final scan stop was caused by insufficient simulated object coverage:

```
ADAPTIVE_SCANNING_WS_ROOT=/tmp/adaptive-scanning-ws.fkcXXi \  
SIM_PART=10 \  
SIM_OBJ_SCALE=0.002 \  
SIM_GZ_GUI=false \  
./sim/sim.sh up
```

The scale override was used only as a local diagnostic. It was not committed as a product default.

## 4.6 Inspect the live ROS graph and logs

```
source /tmp/adaptive-scanning-ws.fkcXXi/install/setup.bash  
  
ros2 action list -t | sort  
ros2 node list | sort  
  
./sim/sim.sh status  
./sim/sim.sh logs gazebo  
./sim/sim.sh logs moveit  
./sim/sim.sh logs scan
```

## 4.7 Stop the simulation

```
ADAPTIVE_SCANNING_WS_ROOT=/tmp/adaptive-scanning-ws.fkcXXi \  
./sim/sim.sh down
```

# 5. PostgreSQL Persistence Test

A temporary PostgreSQL 15 container was used to test the migration against a real database rather than a mock or SQLite compatibility layer.

```
docker run --rm -d \  
  --name armx-fleet-health-test \  
  -e POSTGRES_PASSWORD=testpass \  
  -e POSTGRES_DB=armx_test \  
  -p 127.0.0.1:55432:5432 \  
  postgres:15-alpine
```

The Python smoke test connected with:

```
postgresql://postgres:testpass@127.0.0.1:55432/armx_test
```

It verified:

- application of the dedicated fleet-health migration;
- insertion and retrieval of an active warning condition;
- insertion of an event into the persistent outbox;
- duplicate-safe conflict handling;
- retrieval of pending events;
- event acknowledgement and removal from the pending result;

- runtime-snapshot update; and
- rendering of the PostgreSQL-backed fleet-health viewer.

The temporary database was removed after validation:

```
docker stop armx-fleet-health-test
```

## 6. Test Results

| Test area                              | Result                     | Evidence                                                                                          |
|----------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------|
| Five-package clean build               | Passed                     | All five packages completed the Colcon build and installation stages                              |
| Common package installation            | Passed                     | No missing <code>include/</code> or <code>srv/</code> directory failure                           |
| PostgreSQL fleet-health migration      | Passed                     | Schema and migration record created successfully                                                  |
| Fleet-health condition and outbox flow | Passed                     | Condition write/read, pending event, acknowledgement, and snapshot operations succeeded           |
| Gazebo world and time                  | Passed                     | Dataset object loaded and <code>/clock</code> advanced monotonically                              |
| Simulated robot                        | Passed                     | Robot description loaded and <code>/dsr01/joint_states</code> was published                       |
| Simulated depth camera                 | Passed                     | Camera info, depth image, and point-cloud data were published                                     |
| Controllers                            | Passed                     | Joint-state broadcaster and MoveIt controller became active                                       |
| MoveIt ownership                       | Passed                     | One <code>/dsr01/move_action</code> server was present                                            |
| Robot-owned actions                    | Passed                     | Joint, waypoint, look-at, hand-eye, and parallel-planning interfaces were available as applicable |
| Planning scene                         | Passed                     | Table, robot mount, table stand, and dataset object were available                                |
| Scan startup                           | Passed                     | Common scan nodes started and the robot moved to the scan pose                                    |
| Probe motion                           | Passed                     | All 6 of 6 probe waypoints completed                                                              |
| Full scan completion                   | Stopped by validation gate | Simulated occupied-voxel count remained below the configured minimum                              |

## 7. Remaining Simulation Result

The normal-scale part produced 45 occupied voxels after probing. The diagnostic 2× scale produced 122 occupied voxels. Both results were below the configured minimum of 200, so the scan correctly stopped before look-at scanning and reconstruction continued.

```
Normal scale:    occupied_voxels=45,  required_min=200
Diagnostic scale: occupied_voxels=122, required_min=200
```

This is a simulation perception/configuration result, not a package-build, ROS-interface, controller, MoveIt, or PostgreSQL migration failure. The production threshold was intentionally preserved. Resolving full scan completion requires a separate decision on simulated object placement, scale, camera view, probe coverage, or simulation-specific validation parameters.

## 8. Commits Created

| Repository         | Commit  | Description                                                           |
|--------------------|---------|-----------------------------------------------------------------------|
| Dyne-vision-common | 82e5ace | Adopt PostgreSQL fleet-health persistence and fix Common installation |
| Dyne-vision-robot  | f90bc69 | Restore Gazebo robot runtime startup                                  |

The commits were created locally on `dependency_fix`. They were not pushed during this work.

## 9. Conclusion

The final PostgreSQL fleet-health persistence design is integrated and validated against PostgreSQL 15. All five split-stack ROS packages build successfully in an isolated workspace. The simulation loads the robot, part, camera, controllers, MoveIt, planning scene, and scan pipeline, and it executes the complete probe motion sequence.

The remaining full-scan stop is an explicit occupied-voxel quality gate in the simulation path. No production safety or perception threshold was reduced as part of this migration.